DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 1

Design, Analyse and Validate an Efficient Search Strategy
for a Large-Scale Student Record System

Course Outcome: CO1 – Algorithm fundamentals, efficiency analysis, asymptotic analysis and recurrence analysis.

Algorithms compared

1. Linear Search
2. Recursive Binary Search
3. Hash Table Search

Prepared in a clear, task-by-task format

TASK 1 – Design the Algorithm (5 Marks)

Approach 1: Linear Search (Iterative)

Basic idea: Check registration numbers one by one until the required number is found or the list ends.

Strategy: Brute force. **Preprocessing:** None; the records may be unsorted.

```
LINEAR_SEARCH(A, key)
    for i = 0 to length(A) - 1
        if A[i] == key
            return i
    return -1
```

Why selected: It is simple and works on unsorted data. It provides a baseline for comparison.

Approach 2: Recursive Binary Search

Basic idea: The records must be sorted. Check the middle item and recursively search only the left or right half.

Strategy: Divide and conquer. **Preprocessing:** Sorting costs $\Theta(n \log n)$ if the data is not already sorted.

```
BINARY_SEARCH(A, key, low, high)
    if low > high
        return -1
    mid = (low + high) // 2
    if A[mid] == key
        return mid
    if key < A[mid]
        return BINARY_SEARCH(A, key, low, mid - 1)
    return BINARY_SEARCH(A, key, mid + 1, high)
```

Why selected: It reduces the search range by half at every recursive call.

Approach 3: Hash Table Search

Basic idea: Store each registration number as a key in a hash table so a record can usually be found directly.

Strategy: Hashing. **Preprocessing:** Building the table takes expected $\Theta(n)$ time.

```
BUILD_HASH(A)
    create empty table H
    for each record in A
        H[record.registration_number] = record
    return H

HASH_SEARCH(H, key)
    if key exists in H
        return H[key]
    return NOT_FOUND
```

Why selected: It is suitable for frequent exact-key searches when records change infrequently.

TASK 2 – Identify the Basic Operation (2 Marks)

Algorithm	Basic operation	Reason
Linear Search	Compare A[i] with key	The comparison repeats for each record until a match is found.
Recursive Binary Search	Compare key with A[mid]	One middle comparison is performed per recursive level.
Hash Table	Hash lookup and key equality check	The hash chooses a bucket; average lookup is constant when collisions are controlled.

TASK 3 – Derive the Time-Complexity Function Mathematically (6 Marks)

1. Linear Search

Let n be the number of records. If the key is at position k (positions start at 1), the comparison count is C(n) = k. If the key is absent, C(n) = n. Therefore, T(n) = aC(n) + b, where a is the cost of one comparison and b is fixed overhead. In the worst case, T(n) = an + b, so the running time is $\Theta(n)$. There is no preprocessing cost.

2. Recursive Binary Search

Each call performs constant work and leaves at most n/2 records to search. Thus T(n) = T(n/2) + c, with T(1) = d. After j calls, the subproblem size is n/2■. Stop when n/2■ ≤ 1, which gives j ≥ log■ n. Hence the number of comparisons is at most ■log■ n■ + 1 and the search time is Θ(log n). If sorting is required, total preparation plus one search is Θ(n log n) + Θ(log n) = Θ(n log n).

3. Hash Table Search

Building the table inserts n records, so expected build time is Tbuild(n) = an + b = Θ(n). Once built, a lookup takes expected constant time, Θ(1), with a good hash function and a controlled load factor. For q searches after one build, total expected time is Θ(n + q). With severe collisions, a lookup can take Θ(n) in the worst case.

TASK 4 – Best, Average and Worst-Case Behaviour (4 Marks)

Algorithm	Best case	Average case	Worst case
Linear Search	First item matches: 1 comparison, Θ(1).	Successful key equally likely at any position: about (n+1)/2 comparisons, Θ(n).	Last item or absent: n comparisons, Θ(n).
Binary Search	Middle item matches: Θ(1).	About log■ n comparisons, Θ(log n).	At most ■log■ n■+1 comparisons, Θ(log n).
Hash Table	Expected direct lookup, Θ(1).	Expected Θ(1) with well-distributed keys.	Θ(n) if many keys collide.

The difference occurs because linear search scans records, binary search halves the search interval, and hash-table performance depends on collision distribution.

TASK 5 – Perform Asymptotic Analysis (3 Marks)

Algorithm	Best case	Average / expected	Worst case
Linear Search	Ω(1)	Θ(n)	O(n)
Recursive Binary Search	Ω(1)	Θ(log n)	O(log n)
Hash Table Search	Ω(1)	Θ(1) expected	O(n)

When n grows from 1,000 to 1,000,000, linear search may require one million comparisons. Binary search requires about 10 comparisons for 1,000 records and about 20 for 1,000,000. Hash lookup is expected to remain nearly constant after the table is built. Preprocessing must still be counted.

TASK 6 – Formulate the Recurrence Relation (3 Marks)

For recursive binary search, the original input contains n records. One recursive call is made on at most n/2 records. The work outside the recursive call (finding the middle and comparing values) is constant.

Recurrence: T(n) = T(■n/2■) + c, for n > 1.

Base case: T(1) = d. An empty search range returns immediately.

There is one recursive call per level, and the problem size is halved each time.

TASK 7 – Solve the Recurrence Relation (4 Marks)

Use the iteration method because each call halves the problem size.

```
T(n) = T(n/2) + c
     = T(n/4) + 2c
     = T(n/8) + 3c
     = T(n/2^k) + kc

Stop when n/2^k <= 1
So k >= log2(n)
```

```
T(n) = T(1) + c log2(n)
T(n) = Θ(log n)
```

Therefore, recursive binary search has logarithmic time complexity. Doubling n adds about one comparison level.

TASK 8 – Implement and Test the Algorithms (4 Marks)

This Python code implements all three approaches. It uses the same registration-number list for comparison.

```
def linear_search(a, key):
    for i, value in enumerate(a):
        if value == key:
            return i
    return -1

def binary_search(a, key, low, high):
    if low > high:
        return -1
    mid = (low + high) // 2
    if a[mid] == key:
        return mid
    if key < a[mid]:
        return binary_search(a, key, low, mid - 1)
    return binary_search(a, key, mid + 1, high)

def build_hash(a):
    return {value: i for i, value in enumerate(a)}

def hash_search(h, key):
    return h.get(key, -1)

records = list(range(1, 1001))
h = build_hash(records)
print(linear_search(records, 1000))
print(binary_search(records, 1000, 0, len(records)-1))
print(hash_search(h, 1000))
```

Expected output: all three return 999 for key 1000. All return -1 if the key is absent.

Test cases: first record, middle record, last record, absent registration number, and datasets of 1,000, 10,000, 100,000 and 1,000,000 records.

TASK 9 – Record Experimental Performance (3 Marks)

The measurements below were collected while generating this report. The target was the last item in each dataset. Therefore, linear search makes n comparisons. Timings can vary by computer and should be rerun locally if required.

n	Linear comparisons	Binary comparisons	Linear time (ms)	Binary time (ms)	Hash build (ms)	Hash lookup (ms)
1,000	1,000	10	0.0438	0.0019	0.054	0.0002
10,000	10,000	14	0.5014	0.0021	0.566	0.0001
100,000	100,000	17	5.1766	0.0026	6.939	0.0002
1,000,000	1,000,000	20	52.9234	0.0031	73.024	0.0002

Interpretation: linear comparisons grow directly with n; binary comparisons grow slowly; hash construction grows with n but subsequent lookup is very fast.

TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)

• The observed comparison counts match theory: linear search makes n comparisons for the last record, while binary search uses logarithmic comparisons.

• Binary search and hash lookup scale better than linear search for repeated searches.

• Actual time differs because of Python interpreter overhead, memory allocation, cache effects, and system load.

• Hash-table build time is a separate preprocessing cost. It is more worthwhile when many searches follow the build.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

For frequent exact registration-number searches, I recommend a hash table. It takes expected $\Theta(n)$ time to build and expected $\Theta(1)$ time per lookup. Because records are updated only occasionally, the build cost can be spread over many searches.

Binary search is a good alternative if the records must remain sorted or sorted reports are also needed. It offers $\Theta(\log n)$ lookup but requires sorted data. Linear search is simple but unsuitable for frequent searches over one million records.

The hash table needs extra memory and careful collision handling. The recommendation assumes registration numbers are unique and the table is appropriately sized.

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

A. Impact Analysis (2 Marks)

The system now has 1,000,000 records and must handle 50,000 searches per hour, approximately 13.9 searches per second on average. Records are added only once per day. Searching is frequent compared with updating, so preprocessing is justified.

B. Evaluation of the Original Design (2 Marks)

Linear search may inspect up to 1,000,000 records for one request. Binary search needs about 20 comparisons but requires sorted data. A hash table offers expected constant-time lookup and is suitable when daily updates are infrequent.

C. Adaptation or Redesign (2 Marks)

Decision: Modify the design to use a hash table. Build or update the table after the daily batch of new records. For each search, hash the registration number and retrieve the record. Expected build time is $\Theta(n)$, expected lookup is $\Theta(1)$, and expected total work for q searches after building is $\Theta(n + q)$.

D. Evidence-Based Defence (2 Marks)

For one million records, binary search needs about 20 comparisons per request, while linear search can need one million. Hash lookup is expected to take constant time after $\Theta(n)$ construction. For 50,000 searches per hour, the expected work is $\Theta(n + q)$ after one build rather than $\Theta(qn)$ for repeated linear scans. The university should monitor memory use, collision rate and response time.

Final conclusion: A hash table is the preferred design for repeated exact registration-number searches under the changed requirements. Binary search remains useful when sorted order is also required.